Shift-Left Testing: European Investment Bank Case Study

Software Engineering – Continuous Assessment Report

Prepared by: Gagan Sharma (202401100200140)

1. Introduction

This report examines a case study of the European Investment Bank (EIB), which improved software quality by bringing testing earlier into the development process. The approach is known as shift-left testing: defects are detected as early as possible rather than being left until the end of a project.

In the EIB case, the focus was automated regression testing integrated into the CI/CD (Continuous Integration / Continuous Delivery) pipeline. The reported outcome was an 80% reduction in testing time and more than $700,000 in annual savings. The report explains the problem the approach addresses, how it works, the results reported, and the practical considerations teams should keep in mind.

2. The Problem: Testing Too Late

When testing is concentrated near the end of a project, defects tend to be expensive and time-consuming to fix. Three issues stand out:

- **Late defect discovery.** Bugs may pass through several development stages before anyone finds them, which makes the root cause harder to trace.
- **Slow feedback.** Developers may wait a long time to learn whether a change has broken existing features.
- **Release pressure.** A large volume of testing close to release can delay delivery and compress the time available to fix problems.

The underlying principle is simple: finding a problem earlier usually makes it easier to understand and fix.

3. What Is Shift-Left Testing?

In a traditional workflow, the stages run in sequence (Plan, Code, Integrate, Test) and testing happens late, after most of the work is done. In the shift-left approach, testing starts earlier and continues throughout development (Plan, Code, Test, Integrate, Release). The name comes from moving testing to the left on a project timeline, closer to the start.

Shift-left testing is therefore not only about testing earlier. It is about making quality part of everyday development work rather than a separate phase at the end.

4. How the Approach Works

The EIB case follows a repeatable cycle built around automated regression testing, which checks that new changes have not broken existing functionality.

1. **Write or update code.** A developer makes a change to the application.

2. **Run automated tests.** Regression tests check whether existing features still work.
3. **Use CI/CD quality gates.** Tests run as part of the integration and delivery pipeline, so a change must pass them to move forward.
4. **Fix issues early.** Failures are investigated before the change moves further along the pipeline.

Because this cycle runs on every change, feedback arrives in minutes or hours rather than weeks.

5. Reported Results

The case study reports the following outcomes:

Metric	Reported result
Reduction in testing time	80%
Testing hours	2,000 reduced to 400
Annual savings	More than $700,000

These figures come from the provided case-study handout and are presented here as case-study outcomes, not as independently verified measurements.

6. Evaluation: Benefits and Practical Considerations

Benefits

- Earlier feedback on whether a change is safe.
- Faster regression checks, since they are automated.
- Greater confidence when making changes.
- Closer collaboration, with developers and QA working together throughout development.

Practical considerations

- **Initial effort.** Automated tests take time to design, build, and maintain, so the savings come after an upfront investment.
- **Test quality.** Automation is only valuable when the tests are reliable and relevant. Flaky or outdated tests can slow teams down and erode trust in the pipeline.

7. Conclusion

The EIB case study shows how moving automated regression testing into the CI/CD pipeline can reduce testing effort and support faster, more reliable delivery. Quality should be built in from the beginning rather than inspected at the end.

Key takeaways:

- Test early and repeatedly.
- Automate regression checks where practical.
- Use CI/CD quality gates to catch issues before release.

References

Software Engineering Industry Case Studies handout, "European Investment Bank" entry, pages 1–2. Source listed in the handout: "Total Shift Left: Case Study."